```
import numpy as np
import matplotlib.pyplot as plt
```

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def derSigmoid(x):
    return x * (1 - x)

def tanh(x):
    return np.tanh(x)

def derTanh(x):
    return 1 - np.tanh(x) ** 2

def relu(x):
    return np.maximum(0, x)

def derRelu(x):
    return np.where(x > 0, 1, 0)

def my_func(x):
    return np.sin(x)

def derMy_func(x):
    return np.cos(x)
```

```
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])
```

```python
def calculate_accuracy(predictions, Y):
    preds = (predictions > 0.5).astype(int)  # Округлення до 0 або 1
    return np.mean(preds == Y)  # Підрахунок відсотка правильних прогнозів


def train_network(activation_func, activation_derivative, epochs=10000,
lr=0.1):
    np.random.seed(42)
    input_dim = 2
    hidden_dim = 4
    output_dim = 1

    W1 = np.random.randn(input_dim, hidden_dim) #ваги (коеф)
    b1 = np.zeros((1, hidden_dim)) #зміщення
    W2 = np.random.randn(hidden_dim, output_dim)
    b2 = np.zeros((1, output_dim))

    losses = []
    accuracies = []  # Додаємо список для збереження точності

    for epoch in range(epochs):
        Z1 = np.dot(X, W1) + b1
        A1 = activation_func(Z1)
        Z2 = np.dot(A1, W2) + b2
        A2 = sigmoid(Z2)

        loss = np.mean((y - A2) ** 2)
        losses.append(loss)

        dA2 = (A2 - y) * derSigmoid(A2)
        dW2 = np.dot(A1.T, dA2)
        db2 = np.sum(dA2, axis=0, keepdims=True)

        dA1 = np.dot(dA2, W2.T) * activation_derivative(A1)
        dW1 = np.dot(X.T, dA1)
        db1 = np.sum(dA1, axis=0, keepdims=True)

        W2 -= lr * dW2
        b2 -= lr * db2
        W1 -= lr * dW1
        b1 -= lr * db1

        # Кожні 1000 епох виводимо втрати та точність
        if epoch % 1000 == 0:
            predictions = (A2 > 0.5).astype(int)  # Округлення виходу
            acc = calculate_accuracy(predictions, y)  # Обчислення
точності
            accuracies.append(acc)
            print(f"Epoch {epoch}, Loss: {loss:.4f}, Accuracy: {acc:.4f}")
```

```
    return W1, b1, W2, b2, losses, accuracies
activation_functions = {
    "Sigmoid": (sigmoid, derSigmoid),
    "Tanh": (tanh, derTanh),
    "ReLU": (relu, derRelu),
    "Custom (Sin)": (my_func, derMy_func)
}

results = {}
for name, (act_func, act_deriv) in activation_functions.items():
    print(f"\n--- Навчання з функцією активації: {name} ---")
    W1, b1, W2, b2, losses, accuracies = train_network(act_func,
act_deriv)
    results[name] = {'W1': W1, 'b1': b1, 'W2': W2, 'b2': b2, 'losses':
losses, 'accuracies': accuracies}
    print(f"Фінальна втрата для {name}: {losses[-1]:.5f}")









def predict(X, W1, b1, W2, b2, activation_func):
    Z1 = np.dot(X, W1) + b1
    A1 = activation_func(Z1)
    Z2 = np.dot(A1, W2) + b2
    A2 = sigmoid(Z2)
    return (A2 > 0.5).astype(int)

def plot_decision_boundary(X, y, W1, b1, W2, b2, activation_func):
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
np.linspace(y_min, y_max, 100))
    Z = predict(np.c_[xx.ravel(), yy.ravel()], W1, b1, W2, b2,
activation_func)
    Z = Z.reshape(xx.shape)
    plt.contourf(xx, yy, Z, alpha=0.3)
    plt.scatter(X[:, 0], X[:, 1], c=y.flatten(), edgecolors='k',
cmap=plt.cm.Spectral)
    plt.title(f"Межа прийняття рішення для {activation_func.__name__}")
    plt.show()
```

```
# Візуалізація для кожної функції активації
for name, res in results.items():
    print(f"\n--- Візуалізація межі прийняття рішення для {name} ---")
    plot_decision_boundary(X, y, res['W1'], res['b1'], res['W2'],
res['b2'], activation_functions[name][0])

# Візуалізація графіків втрат для кожної активаційної функції
for name, res in results.items():
    plt.figure(figsize=(6, 4))
    plt.plot(res['losses'], label=f"{name}")
    plt.xlabel("Epochs")
    plt.ylabel("Loss")
    plt.title(f"Зміна втрат під час навчання ({name})")
    plt.legend()
    plt.grid()
    plt.show()
```

Що виправити?

1. **Опис функцій активації (ReLU не завжди 0)**.
2. **Що таке "зміщення" (bias) і чому воно потрібне**.
3. **Чому sigmoid() тільки на вихідному шарі**.
4. **Як працює backpropagation (три етапи)**.
5. **Що таке Loss і Accuracy (чіткіше пояснити)**.
6. **Графіки – пояснити, що вони показують**.

Значить у мене перша лабораторна робота. У ній потрібно було реалізувати нейронну мережу прямого звязку. Навчити мережу на наборі даних проблема XOR і візуалізувати межу рішення. Треба було використовувати функції активації ReLU, Sigmoid, Tanh та кастомну функцію.

Значить спочатку я імпортував бібліотеки, відповідно для обрахунків і виводу графіків.

```
import numpy as np
import matplotlib.pyplot as plt
```

Далі прописав функції активації та їх похідні бо без похідної нейромережа не знає, наскільки сильно змінювати ваги.

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def derSigmoid(x):
    return x * (1 - x)

def tanh(x):
    return np.tanh(x)

def derTanh(x):
    return 1 - np.tanh(x) ** 2

def relu(x):
    return np.maximum(0, x)

def derRelu(x):
    return np.where(x > 0, 1, 0)
```

Тут слід наголосити увагу на функції sigmoid, оскільки Диференційована функція, підходить для ймовірнісних задач, оскільки надає значення між 0 та 1, для tanh(x) Диференційована функція має значення між -1 та 1, (корисна при роботі з центрально-симетричними даними. для def relu(x): **обнуляє всі від’ємні значення**, але пропускає **позитивні значення без змін**.

Далі йде кастомна функція, я обрав

```
def my_func(x):
    return np.sin(x)

def derMy_func(x):
    return np.cos(x)
```

Далі у мене прописаний набір даних XOR,
на вході , тобто X я маю
([[0, 0], [0, 1], [1, 0], [1, 1]]), на виході тобто Y я маю ([[0], [1], [1], [0]]).
Чому саме такі значення, бо у нас проблема XOR, і якщо на вхід поступає
0 0 то відповідно ма виході маємо 0, якщо 0 1 то на виході 1, 1 0 то на виході 1 і якщо 1 1 то на виході відповідно 0.

Далі йде реалізація мережі та алгоритму.
Тут у мене прописані дві функції. Перша
def calculate_accuracy(predictions, Y):
    preds = (predictions > 0.5).astype(int)
    return np.mean(preds == Y)
Функція яка рахує точність моделі.

Друга це def train_network.
Дана функція приймає на вхід функцію активації та її похідну. epochs=10000 це кількість ітерацій для навчання а lr=0.1 це коефіцієнт швидкості навчання необхідний для градієнтного спуску.

спочатку у мене прописані змінні це
np.random.seed(42) - для того щоб генерувалося постійно одні й ті самі числа
input_dim = 2 - кількість входів
hidden_dim = 4 - кількість нейронів у прихованому шарі **Для проблеми XOR мінімальна необхідна кількість нейронів у прихованому шарі – 2** (чим їх більше тим точніше обрахунки, але я обрав просто в 2 рази більшу кулькість і для 4 в мене там достатня точність обрахунків)
output_dim = 1 - кількість виходів

Значить далі йде ініціалізація початкових вагів (тобто це коефіцієнти) та зміщення (воно **додає додаткову гнучкість нейронній мережі** і Без нього модель буде **завжди прив'язана до початку координат** (0,0).)
W1 = np.random.randn(input_dim, hidden_dim) (це ваги між вхідним та прихованим шаром)
b1 = np.zeros((1, hidden_dim)) Зміщення для прихованого шару
W2 = np.random.randn(hidden_dim, output_dim) Ваги між прихованим і вихідним шарами
b2 = np.zeros((1, output_dim)) Зміщення для вихідного шару

Ваги (W1, W2) визначають, як сильно кожен нейрон впливає на наступний шар.
Зміщення (b1, b2) додають додаткову гнучкість, щоб модель не прив'язувалася до нуля.

Далі прописані 2 масиви

losses = [] (**наскільки сильно передбачення відрізняються від правильних відповідей**.)
accuracies = [] (точність відповідей моделі)

де ми зберігаємо значення втрат та точності відповідно.

Далі йде цикл for epoch in range(epochs):, який повторюється 10 000 разів.
У самому циклі спочатку прописано пряме поширення (forward pass):
Це коли вхідні дані **проходять через нейромережу** і видають передбачення (A2).
Z1 = np.dot(X, W1) + b1 (лініцне перетворення входів)
A1 = activation_func(Z1) (функція активації залежно від вхідних даних)
Z2 = np.dot(A1, W2) + b2 (вагове перетворення виходу з прихованого шару)
A2 = sigmoid(Z2) (остаточний результат (Sigmoid для отримання ймовірностей між 0 і 1).)

Далі у нас обраховується функція втрат MSE

```
loss = np.mean((y - A2) ** 2)
losses.append(loss)
```

Ну і відповідно далі прописане зворотне поширення (Backpropogation) це **метод оновлення ваг** шляхом передачі помилки **з вихідного шару назад у прихований**.
dA2 = (A2 - y) * derSigmoid(A2) **тут виконується Обчислення градієнта для вихідного шару, Помилка (A2 – y) показує, наскільки передбачене значення A2 відрізняється від справжнього. Множимо її на derSigmoid(A2), бо похідна Sigmoid контролює, як змінювати ваги.**

dW2 = np.dot(A1.T, dA2) (обчислюємо градієнт ваг для W2.)
db2 = np.sum(dA2, axis=0, keepdims=True) (градієнт для зміщення b2)

dA1 = np.dot(dA2, W2.T) * activation_derivative(A1) (**Обчислення градієнта для прихованого шару**) Помилка **розповсюджується назад** через ваги W2 і Множимо на похідну функції анктивації, бо це визначає, **наскільки сильно прихований шар вплинув на помилку**.

dW1 = np.dot(X.T, dA1) (обчислюємо градієнт ваг для W1)
db1 = np.sum(dA1, axis=0, keepdims=True) (градієнт для зміщення b1.)

Після чого прописаний градієнтний спуск (тобто це такий певний процес зміни ваг **у правильному напрямку**, щоб **зменшити помилку в наступних**

ітераціях.), тобто ми робимо оновлення ваг та зміщень **Градієнтний спуск** – це **метод оновлення ваг** у нейронній мережі.

```
W2 -= lr * dW2
b2 -= lr * db2
W1 -= lr * dW1
b1 -= lr * db1
```

Ну і відповідно далі виконується перевірка точності кожні 1000 епох

```
if epoch % 1000 == 0:
        predictions = (A2 > 0.5).astype(int)
        acc = calculate_accuracy(predictions, y)
        accuracies.append(acc)
        print(f"Epoch {epoch}, Loss: {loss:.4f}, Accuracy: {acc:.4f}")
```

Так як за умовою завдання нам потрібно було протестувати 4 функції активації а саме "Sigmoid" Tanh "ReLU" Custom (Sin)". Я створив словник

```
activation_functions = {
   "Sigmoid": (sigmoid, derSigmoid),
   "Tanh": (tanh, derTanh),
   "ReLU": (relu, derRelu),
   "Custom (Sin)": (my_func, derMy_func)
}
```

та словник результатів results = {}

ну і просто пройшовся циклом for name, (act_func, act_deriv) in activation_functions.items(): по цьому словнику і для кожної функції активації використав функцію train_network(act_func, act_deriv)

Ну і відповідно маю такі результати

```
Навчання з функцією активації: Sigmoid
Epoch 0, Loss: 0.2832, Accuracy: 0.5000
Epoch 1000, Loss: 0.2452, Accuracy: 0.7500
Epoch 2000, Loss: 0.2124, Accuracy: 0.7500
Epoch 3000, Loss: 0.1503, Accuracy: 0.7500
Epoch 4000, Loss: 0.0572, Accuracy: 1.0000
Epoch 5000, Loss: 0.0209, Accuracy: 1.0000
Epoch 6000, Loss: 0.0107, Accuracy: 1.0000
Epoch 7000, Loss: 0.0067, Accuracy: 1.0000
Epoch 8000, Loss: 0.0047, Accuracy: 1.0000
Epoch 9000, Loss: 0.0035, Accuracy: 1.0000
Фінальна втрата для Sigmoid: 0.00279
```

Навчання з функцією активації: Tanh

Epoch 0, Loss: 0.3223, Accuracy: 0.5000
Epoch 1000, Loss: 0.0200, Accuracy: 1.0000
Epoch 2000, Loss: 0.0045, Accuracy: 1.0000
Epoch 3000, Loss: 0.0023, Accuracy: 1.0000
Epoch 4000, Loss: 0.0015, Accuracy: 1.0000
Epoch 5000, Loss: 0.0011, Accuracy: 1.0000
Epoch 6000, Loss: 0.0009, Accuracy: 1.0000
Epoch 7000, Loss: 0.0007, Accuracy: 1.0000
Epoch 8000, Loss: 0.0006, Accuracy: 1.0000
Epoch 9000, Loss: 0.0005, Accuracy: 1.0000
Фінальна втрата для Tanh: 0.00048

Навчання з функцією активації: ReLU
Epoch 0, Loss: 0.3554, Accuracy: 0.5000
Epoch 1000, Loss: 0.0144, Accuracy: 1.0000
Epoch 2000, Loss: 0.0034, Accuracy: 1.0000
Epoch 3000, Loss: 0.0018, Accuracy: 1.0000
Epoch 4000, Loss: 0.0012, Accuracy: 1.0000
Epoch 5000, Loss: 0.0009, Accuracy: 1.0000
Epoch 6000, Loss: 0.0007, Accuracy: 1.0000
Epoch 7000, Loss: 0.0005, Accuracy: 1.0000
Epoch 8000, Loss: 0.0005, Accuracy: 1.0000
Epoch 9000, Loss: 0.0004, Accuracy: 1.0000
Фінальна втрата для ReLU: 0.00035

Навчання з функцією активації: Custom (Sin)
Epoch 0, Loss: 0.3356, Accuracy: 0.5000
Epoch 1000, Loss: 0.0166, Accuracy: 1.0000
Epoch 2000, Loss: 0.0037, Accuracy: 1.0000
Epoch 3000, Loss: 0.0020, Accuracy: 1.0000
Epoch 4000, Loss: 0.0014, Accuracy: 1.0000
Epoch 5000, Loss: 0.0011, Accuracy: 1.0000
Epoch 6000, Loss: 0.0008, Accuracy: 1.0000
Epoch 7000, Loss: 0.0007, Accuracy: 1.0000
Epoch 8000, Loss: 0.0006, Accuracy: 1.0000
Epoch 9000, Loss: 0.0005, Accuracy: 1.0000
Фінальна втрата для Custom (Sin): 0.00048

Тут ми бачимо що на першій епосі для навчання з функцією активації Sigmoid у нас точність 0.5, адже воно просто вгадує, на 1000 - 3000 епохах точність 0.75. Починаючи з 4000 епохи точність 1 (100%) Фінальна втрата для Sigmid становить 0.00279

Ну і аналогічне пояснення для інших функцій активації.

Далі у мене йде побудова графіків функцій межі прийняття рішень та зміни втрат при навчанні

Тут прописана функція def predict, Вона **отримує передбачення нейромережі** (від 0 до 1) для **нових даних**. Використовується у функції plot_decision_boundary(), щоб визначити, де **мережа передбачає клас 0, а де – клас 1**.

```
def predict(X, W1, b1, W2, b2, activation_func):
    Z1 = np.dot(X, W1) + b1
    A1 = activation_func(Z1)
    Z2 = np.dot(A1, W2) + b2
    A2 = sigmoid(Z2)
    return (A2 > 0.5).astype(int)
```

Ну і далі відповідно функція побудови графіків, вона

будує межу прийняття рішення нейромережі.

Фіолетова зона – де **нейромережа передбачає клас 0**.

Жовта зона – де **нейромережа передбачає клас 1**.

Зелена лінія – це **межа, де модель найменш впевнена у рішенні**.

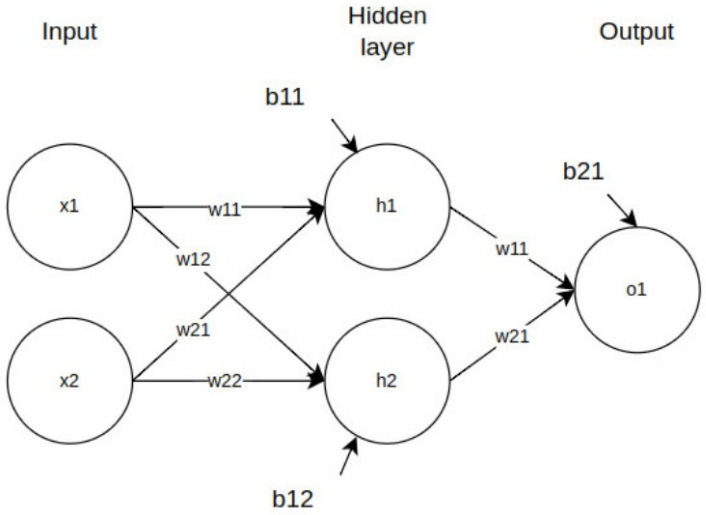
```
def plot_decision_boundary(X, y, W1, b1, W2, b2, activation_func):
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
        xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100), np.linspace(y_min, y_max, 100))
    Z = predict(np.c_[xx.ravel(), yy.ravel()], W1, b1, W2, b2, activation_func)
    Z = Z.reshape(xx.shape)
    plt.contourf(xx, yy, Z, alpha=0.3)
    plt.scatter(X[:, 0], X[:, 1], c=y.flatten(), edgecolors='k', cmap=plt.cm.Spectral)
    plt.title(f"Межа прийняття рішення для {activation_func.__name__}")
    plt.show()


for name, res in results.items():
        plot_decision_boundary(X, y, res['W1'], res['b1'], res['W2'], res['b2'], activation_functions[name][0])

for name, res in results.items():
    plt.figure(figsize=(6, 4))
    plt.plot(res['losses'], label=f"{name}")
    plt.xlabel("Epochs")
    plt.ylabel("Loss")
    plt.title(f"Зміна втрат під час навчання ({name})")
    plt.legend()
    plt.grid()
```

```
plt.show()
```

Персептрон - **найпростіша модель нейронної мережі**, що імітує роботу одного нейрона.



📌 1. Пояснення основних функцій NumPy та Matplotlib

Функція	Що робить?	Приклад
`np.dot(A, B)`	Обчислює **матричний добуток** двох матриць.	`C = np.dot(A, B)`
`np.zeros(shape)`	Створює **матрицю, заповнену нулями**.	`arr = np.zeros((2,3))` → `[[0,0,0], [0,0,0]]`
`np.mean(array)`	Обчислює **середнє значення** елементів масиву.	`avg = np.mean([1, 2, 3])` → `2.0`
`.append(value)`	**Додає елемент** у список.	`arr.append(5)`
`np.sum(array, axis=N)`	**Складає елементи** масиву (за віссю `axis`, якщо вказано).	`np.sum([[1,2],[3,4]], axis=0)` → `[4,6]`
`np.meshgrid(x, y)`	Створює **дві 2D-матриці**, які використовуються для побудови поверхонь або контурних графіків.	`xx, yy = np.meshgrid(np.linspace(-1,1,10), np.linspace(-1,1,10))`
`plt.contourf(X, Y, Z)`	**Створює контурну карту** (фарбує області за рівнями значень `Z`).	`plt.contourf(xx, yy, Z, alpha=0.3)`
`plt.scatter(X, Y, c=color)`	**Малює точки** (X, Y) на графіку з кольором `color`.	`plt.scatter(X[:,0], X[:,1], c=y)`